Algoritmi e Strutture Dati

Capitolo 2
Modelli di calcolo e metodologie di analisi

Camil Demetrescu, Irene Finocchi,
Giuseppe F. Italiano



Notazione asintotica

Copyright © 2008 - The McGraw - Hill Companies, srl



Notazione asintotica

- f(n) = tempo di esecuzione / occupazione di memoria di un algoritmo su input di dimensione n
- La notazione asintotica è un'astrazione utile per descrivere l'ordine di grandezza di f(n) ignorando i dettagli non influenti, come costanti moltiplicative e termini di ordine inferiore

Copyright © 2008 - The McGraw - Hill Companies, srl



Tre notazioni: motivazioni e definizioni

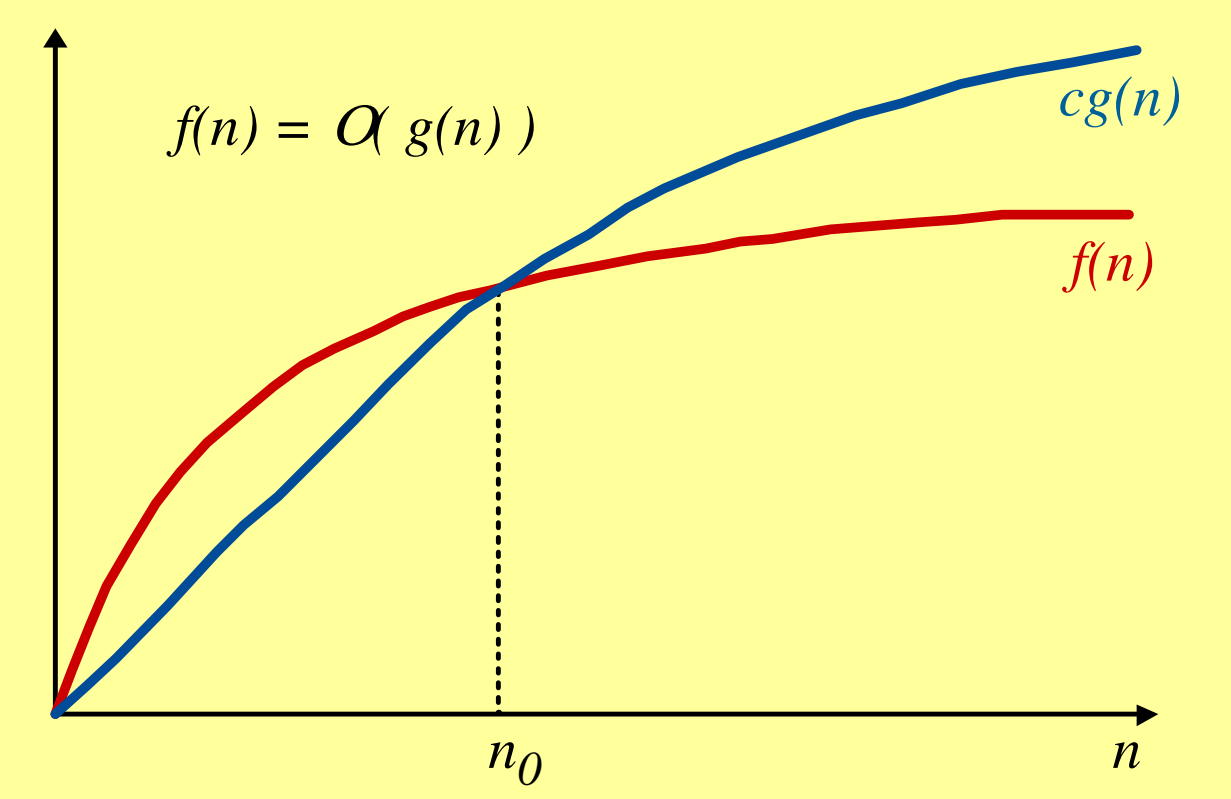
O, Ω, Θ

- **O** e Ω forniscono un limite "lasco", rispettivamente per i limiti superiore ed inferiore

- Θ fornisce un limite "stretto"
  - In alcuni contesti è difficile trovare un limite stretto per l'andamento delle funzioni, per cui ci si accontenta di un limite meno preciso

Notazione asintotica O

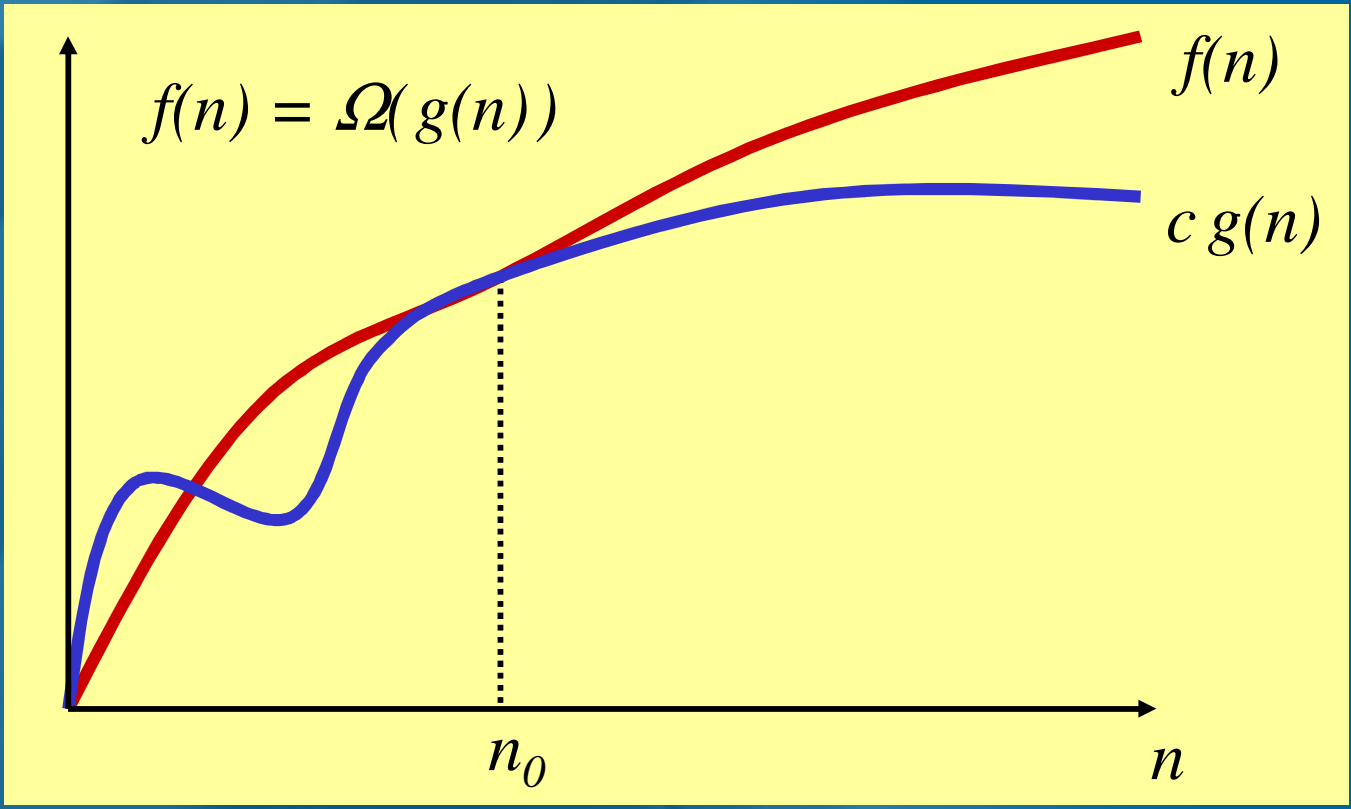
$f(n) = O(g(n))$ se $\exists$ due costanti $c>0$ e $n_0 \geq 0$ tali che $f(n) \leq c\, g(n)$ per ogni $n \geq n_0$



Copyright © 2008 - The McGraw - Hill Companies, srl

Notazione asintotica Ω

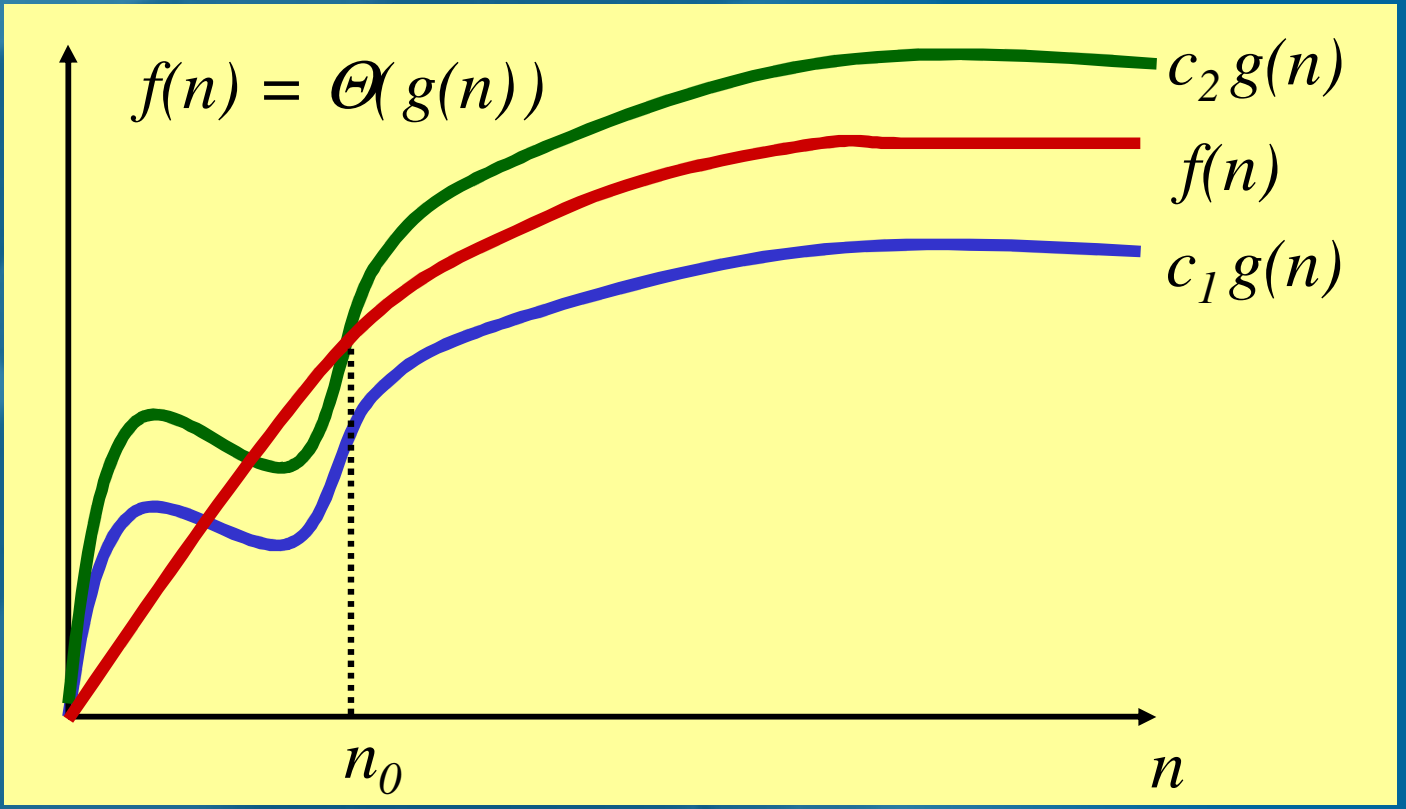
$f(n) = \Omega(g(n))$ se $\exists$ due costanti $c>0$ e $n_0 \geq 0$ tali che $f(n) \geq c\, g(n)$ per ogni $n \geq n_0$



Copyright © 2008 - The McGraw - Hill Companies, srl

Notazione asintotica Θ

$f(n) = \Theta(g(n))$ se $\exists$ tre costanti $c_1, c_2 > 0$ e $n_0 \geq 0$ tali che $c_1\, g(n) \leq f(n) \leq c_2\, g(n)$ per ogni $n \geq n_0$



Copyright © 2008 - The McGraw - Hill Companies, srl



Abuso di notazione

Si noti che usiamo $f(n) = O(g(n))$ invece di $f(n) \in O(g(n))$

$f(n) = O(g(n))$ si legge $f(n)$ è "o grande" di $g(n)$

Se $f(n) = O(g(n))$ è il tempo calcolo richiesto da un algoritmo $\mathcal{A}lg$ diciamo che $O(g(n))$ è un limite superiore asintotico per la complessità tempo di $\mathcal{A}lg$

Similmente per le altre notazioni



Teorema

Dalle definizioni discende il seguente teorema:

Date due funzioni f(n) e g(n), $f(n) \in \Theta(g(n))$ se e solo se $f(n) \in O(g(n))$ e $f(n) \in \Omega(g(n))$

Copyright © 2008 - The McGraw - Hill Companies, srl



Notazione asintotica: esempi

- Sia $g(n)=3n^2+10n$
- $g(n)=O(n^2)$: scegliere $c=4$ e $n_0=10$
- $g(n)=\Omega(n^2)$: scegliere $c=1$ e $n_0=0$
- $g(n)=\Theta(n^2)$: infatti $g(n)= \Theta(f(n))$ se e solo se $g(n)= O(f(n))$ e $g(n)= \Omega(f(n))$
- $g(n)=O(n^3)$ ma $g(n)\neq\Theta(n^3)$

Copyright © 2008 - The McGraw - Hill Companies, srl



Transitività

$$f(n) = \Theta(g(n)) \quad e \quad g(n) = \Theta(h(n)) \quad \Rightarrow \quad f(n) = \Theta(h(n))$$

$$f(n) = O(g(n)) \quad e \quad g(n) = O(h(n)) \quad \Rightarrow \quad f(n) = O(h(n))$$

$$f(n) = \Omega(g(n)) \quad e \quad g(n) = \Omega(h(n)) \quad \Rightarrow \quad f(n) = \Omega(h(n))$$

Copyright © 2008 - The McGraw - Hill Companies, srl



Legge transitiva per O

La relazione "è O grande di" soddisfa la proprietà transitiva, cioè se $f(n)$ è $O(g(n))$ e $g(n)$ è $O(h(n))$ abbiamo anche che $f(n)$ è $O(h(n))$.

$f(n)$ è $O(g(n))$ ➔ $\forall n \geq n_1 \;\; f(n) \leq c_1 g(n)$.

$g(n)$ è $O(h(n))$ ➔ $\forall n \geq n_2 \;\; g(n) \leq c_2 h(n)$.

Basta prendere come $n_0 = \max(n_1, n_2)$ e $c = c_1 \cdot c_2$.

Copyright © 2008 - The McGraw - Hill Companies, srl



Riflessività

$$f(n) = \Theta(f(n))$$

$$f(n) = O(f(n))$$

$$f(n) = \Omega(f(n))$$

Copyright © 2008 - The McGraw - Hill Companies, srl



Confronto tra funzioni : simmetria

$$f(n) = \Theta(g(n)) \quad \Leftrightarrow \quad g(n) = \Theta(f(n))$$

Confronto tra funzioni : simmetria transposta

$$f(n) = O(g(n)) \quad \Leftrightarrow \quad g(n) = \Omega(f(n))$$



Regole di semplificazione

1. Se f(n) =O(k g(n)) con k costante >0, allora **f(n) = O(g(n))**

2. **Somma**. Se f1(n) = O(g1(n)) e f2(n) = O(g2(n)), allora **f1(n) + f2(n) = O(max(g1(n), g2(n)))**
(Blocchi sequenziali di istruzioni, rami if-then-else, switch)

3. **Prodotto**. Se f1(n) = O(g1(n)) e f2(n) = O(g2(n)), allora **f1(n)f2(n) = O(g1(n)g2(n))**
(Cicli for, while, do-while)

Similmente per Ω e Θ

Esempio: Se $T(n) = 3n^4+5n^2$, allora T(n)= ?

Copyright © 2008 - The McGraw - Hill Companies, srl



Regole di semplificazione

1. Se f(n) =O(k g(n)) con k costante >0, allora **f(n) = O(g(n))**

2. **Somma**. Se f1(n) = O(g1(n)) e f2(n) = O(g2(n)), allora **f1(n) + f2(n) = O(max(g1(n), g2(n)))**
(Blocchi sequenziali di istruzioni, rami if-then-else, switch)

3. **Prodotto**. Se f1(n) = O(g1(n)) e f2(n) = O(g2(n)), allora **f1(n)f2(n) = O(g1(n)g2(n))**
(Cicli for, while, do-while)

Similmente per Ω e Θ

Esempio: Se $T(n) = 3n^4+5n^2$, allora $T(n)= O(n^4)$

Copyright © 2008 - The McGraw - Hill Companies, srl



Regole di semplificazione: esercizi

Di che ordine è un algoritmo che ha come funzione T(n):

a. $8n^3 - 9n$
b. $7 \log n + 20$
c. $9 \log n + n$
d. $n \log n + n^2$
e. $\log (\log n) + 3 \log n + 4$

Copyright © 2008 - The McGraw - Hill Companies, srl



Regole di semplificazione: esercizio

Supponi che l'implementazione di un certo algoritmo in Java che processa un array di n elementi sia del tipo:

```
for (int pass = 1; pass <= n; pass++){
  for (int index = 0; index < n; index++)
    for (int count = 1; count < 10; count++)
          . . . //<istruzione non dipendente da n>
}
```

Qual è l'ordine della funzione T(n) dell'algoritmo?

Copyright © 2008 - The McGraw - Hill Companies, srl



Notazione asintotica versus numeri naturali

$$f(n) = O(g(n)) \approx a \leq b$$

$$f(n) = \Omega(g(n)) \approx a \geq b$$

$$f(n) = \Theta(g(n)) \approx a = b$$

Copyright © 2008 - The McGraw - Hill Companies, srl



Proprietà di tricotomia

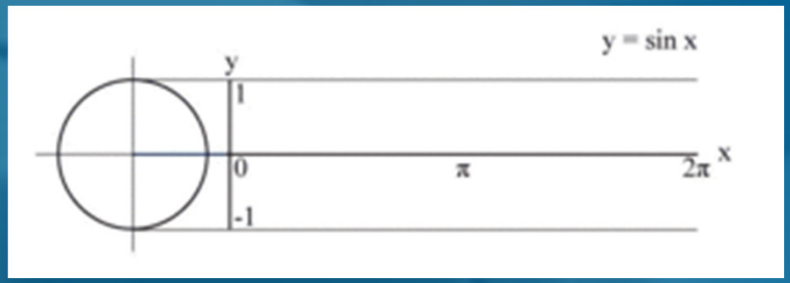
Per ogni coppia di numeri reali a e b, deve valere esattamente una delle seguenti espressioni:

$$a < b, \quad a = b, \quad a > b$$

Sebbene qualunque coppia di numeri possa essere confrontata, non tutte le funzioni sono asintoticamente confrontabili!

Esempio:

$$f(n) = n, \quad g(n) = n^{1+\sin n}$$



Copyright © 2008 - The McGraw - Hill Companies, srl



Teorema dei limiti (1)

Se f(n) e g(n) sono asintoticamente positive, dal limite del loro rapporto è possibile dedurre:

1. $\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0 \quad \Rightarrow \quad f(n) = O(g(n))$

$f(n) \neq \Theta(g(n))$

$f(n) \neq \Omega(g(n))$

Notare che:

$f(n) = O(g(n)) \quad \not\Rightarrow \quad \lim_{n \to \infty} \frac{f(n)}{g(n)} = 0$

$f(n) = O(g(n)) \quad \Rightarrow \quad \lim_{n \to \infty} \frac{f(n)}{g(n)}$ (se esiste) $< \infty$

Copyright © 2008 - The McGraw - Hill Companies, srl



Teorema dei limiti (2)

Se f(n) e g(n) sono asintoticamente positive, dal limite del loro rapporto è possibile dedurre:

2. $\lim_{n \to \infty} \frac{f(n)}{g(n)} = \infty \quad \Rightarrow \quad f(n) = \Omega(g(n))$

$f(n) \neq O(g(n))$

$f(n) \neq \Theta(g(n))$

Notare che:

$f(n) = \Omega(g(n)) \quad \not\Rightarrow \quad \lim_{n \to \infty} \frac{f(n)}{g(n)} = \infty$

$f(n) = \Omega(g(n)) \quad \Rightarrow \quad \lim_{n \to \infty} \frac{f(n)}{g(n)}$ (se esiste) > 0

Copyright © 2008 - The McGraw - Hill Companies, srl



Teorema dei limiti (3)

Se f(n) e g(n) sono asintoticamente positive, dal limite del loro rapporto è possibile dedurre:

3. $\lim_{n \to \infty} \frac{f(n)}{g(n)} \in R^+ \Rightarrow$ $f(n) = \Theta(g(n))$, $g(n) = \Theta(f(n))$

4. $\lim_{n \to \infty} \frac{f(n)}{g(n)}$ non esiste $\Rightarrow$ f(n) e g(n) non sono asintoticamente confrontabili.

Dimostrazione: Segue dalla teoria di Analisi Matematica.

Copyright © 2008 - The McGraw - Hill Companies, srl



Un pò di terminologia

Alcune classi di *complessità asintotica*:

O(1): costante, non dipende dalla dimensione dei dati

O(lg n): logaritmica (in generale: $\lg^k n$ k>=1)

O(n): lineare

O(n lg n): pseudolineare

O(n^2): quadratica (n^k polinomiale k>1)

O(2^n): esponenziale (in generale: k^n k>1)

Copyright © 2008 - The McGraw - Hill Companies, srl

Sommatorie
[vedi dispensa su Sommatorie]

Copyright © 2008 - The McGraw - Hill Companies, srl



Esempi – complessità tempo (1)

Esempio 1: `a = b;`

Tempo costante $\Theta(1)$

Esempio 2:

```
sum = 0;
for (i=1; i<=n; i++)
   sum += n;
```

$\Theta(n)$



Esempi –complessità tempo (2)

Example 3:

```
sum = 0;
for (j=1; j<=n; j++)//Primo for (doppio)
   for (i=1; i<=j; i++)
      sum++;
for (k=0; k<n; k++) //Secondo for
   A[k] = k;
```

serie aritmetica

$\Theta(n^2)$



Esempi – complessità tempo(3)

Esempio 4:

```
sum1 = 0;
for (i=1; i<=n; i++)
   for (j=1; j<=n; j++)
      sum1++;

sum2 = 0;
for (i=1; i<=n; i++)
   for (j=1; j<=i; j++)
      sum2++;
```

serie aritmetica

Entrambi i loop $\Theta(n^2)$



Esempi – complessità tempo(3)

Esempio 5: Assumiamo n potenza del 2

```
sum1 = 0;
for (k=1; k<=n; k*=2) // lg n volte
    for (j=1; j<=n; j++) // n volte
      sum1++;
```

$$T(n) = \sum_{i=0}^{\lg n} n = \Theta(n \lg n)$$



Esempi – complessità tempo(3)

Esempio 5: Assumiamo n potenza del 2

```
sum1 = 0;
for (k=1; k<=n; k*=2) // lg n volte
    for (j=1; j<=n; j++) // n volte
        sum1++;
sum2 = 0;
for (k=1; k<=n; k*=2) // lg n volte
    for (j=1; j<=k; j++) // k volte
        sum2++;
```

$$T(n) = \sum_{i=0}^{\lg n} n = \Theta(n \lg n)$$

$\Theta(n \lg n)$

$$T(n) = \sum_{i=0}^{\lg n} 2^i = \Theta(n)$$

serie geometrica



Altre strutture di controllo

Altri loop: come il `for` loop

`if-then-else`: considera la complessità maggiore (max) delle due clausole `then`/`else`

`switch`: considera la complessità del caso più costoso (max)

Invocazione di una subroutine: complessità della subroutine



Complessità intrinseca di un problema

Copyright © 2008 - The McGraw - Hill Companies, srl



Complessità di un problema

Complessità intrinseca di un problema ≠
Complessità computazionale di un algoritmo

- Definizioni:

Un **problema computazionale** ha **complessità O(f(n))** (**upper bound**) se esiste un algoritmo per la sua risoluzione con delimitazione superiore O(f(n)).

Un **problema computazionale** ha **complessità Ω(f(n))** (**lower bound**) se tutti gli algoritmi per la sua risoluzione hanno delimitazione inferiore Ω(f(n)).

Copyright © 2008 - The McGraw - Hill Companies, srl



Algoritmi ottimali

Se dimostro che un **problema ha delimitazione inferiore Ω(f(n))** e trovo un **algoritmo avente delimitazione superiore O(f(n))** allora, a meno di costanti, ho un algoritmo ottimale per risolvere il problema!!!

Esempio (Ricerca del minimo in un insieme non ordinato): Un algoritmo per la ricerca del minimo in un insieme non ordinato, avente complessità O(n), è ottimo. Infatti, ogni algoritmo dovrà almeno leggere l'input, e quindi il problema avrà complessità inferiore Ω(n).

Copyright © 2008 - The McGraw - Hill Companies, srl



Metodi di analisi

Copyright © 2008 - The McGraw - Hill Companies, srl



Caso peggiore, migliore e medio

- Misureremo le risorse di calcolo usate da un algoritmo (tempo di esecuzione / occupazione di memoria) in funzione della dimensione n delle istanze di input
- Istanze diverse, a parità di dimensione, potrebbero però richiedere risorse diverse
- Distinguiamo quindi ulteriormente tra analisi nel caso peggiore, migliore e medio

Copyright © 2008 - The McGraw - Hill Companies, srl



Caso peggiore

- Sia tempo(I) il tempo di esecuzione di un algoritmo sull'istanza I
- $T_{worst}(n) = \max_{\text{istanze I di dimensione n}} \{\text{tempo}(I)\}$
- Intuitivamente, $T_{worst}(n)$ è il tempo di esecuzione sulle istanze di ingresso che comportano più lavoro per l'algoritmo

Copyright © 2008 - The McGraw - Hill Companies, srl



Caso migliore

- Sia tempo(I) il tempo di esecuzione di un algoritmo sull'istanza I
- $T_{best}(n) = \min_{\text{istanze I di dimensione n}} \{\text{tempo}(I)\}$
- Intuitivamente, $T_{best}(n)$ è il tempo di esecuzione sulle istanze di ingresso che comportano meno lavoro per l'algoritmo

Copyright © 2008 - The McGraw - Hill Companies, srl



Caso medio

- Sia $\mathcal{P}(I)$ la probabilità di avere in ingresso un'istanza I
- $T_{avg}(n) = \sum_{\text{istanze I di dimensione n}} \{ \mathcal{P}(I) \text{ tempo}(I) \}$
- Intuitivamente, $T_{avg}(n)$ è il tempo di esecuzione nel caso medio, ovvero sulle istanze di ingresso "tipiche" per il problema
- Richiede conoscenza di una distribuzione di probabilità sulle istanze

Copyright © 2008 - The McGraw - Hill Companies, srl



Esempio 1

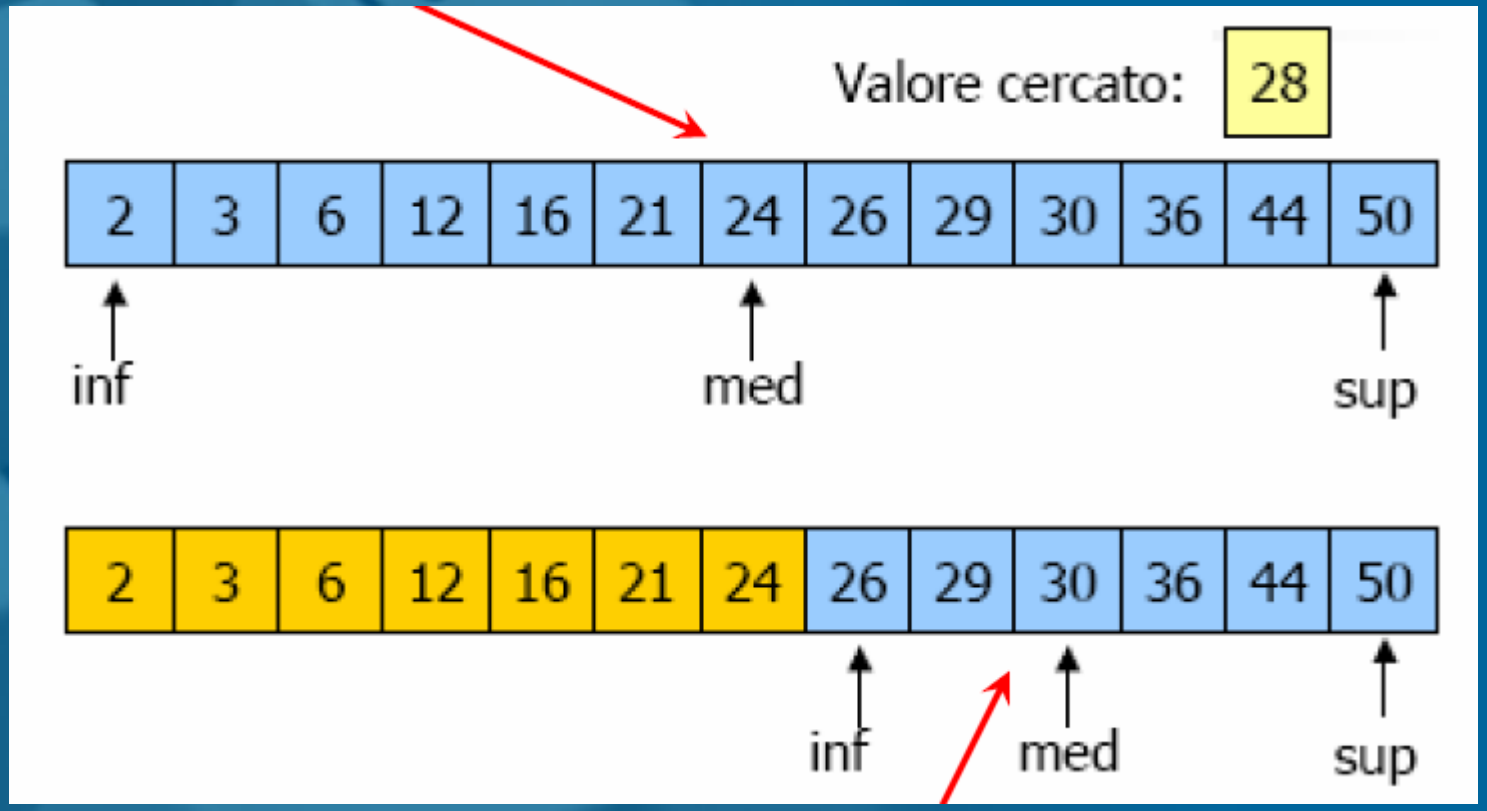
Ricerca di un elemento x in una lista $\mathcal{L}$ non ordinata

```
algoritmo ricercaSequenziale(lista L, elem x) → booleano
1.    for each ( y ∈ L ) do
2.        if ( y = x ) then return trovato
3.    return non trovato
```

- $T_{best}(n) = 1$ x è in prima posizione
- $T_{worst}(n) = n$ $x \notin \mathcal{L}$ oppure è in ultima posizione
- $T_{avg}(n) = (n+1)/2$ Assumendo che $x \in \mathcal{L}$ e che le istanze siano equidistribuite: $p\{pos(x)=i\} = 1/n$ per ogni i in [1,n]

Copyright © 2008 - The McGraw - Hill Companies, srl

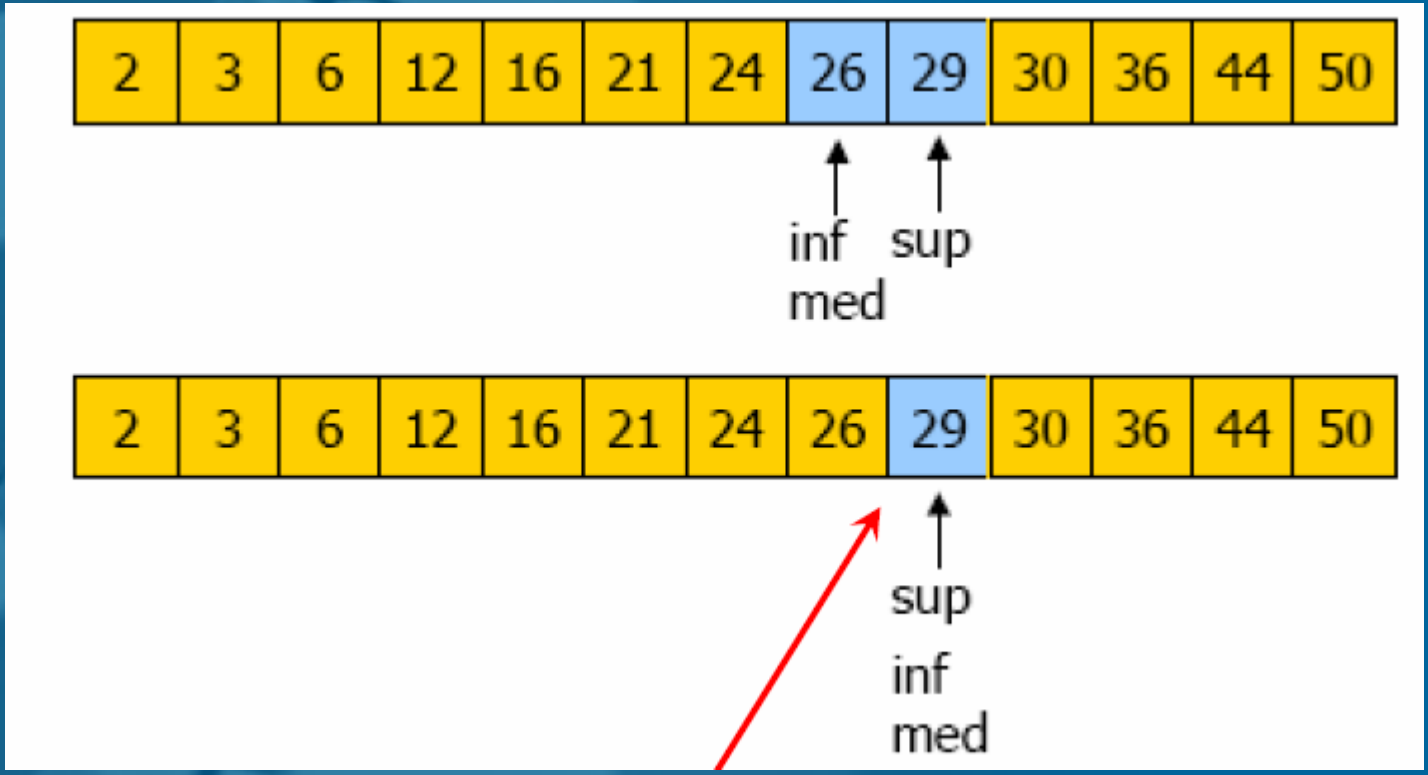
Esempio2: ricerca binaria
(ricerca senza successo)



Assume che la sequenza di elementi sia ordinata

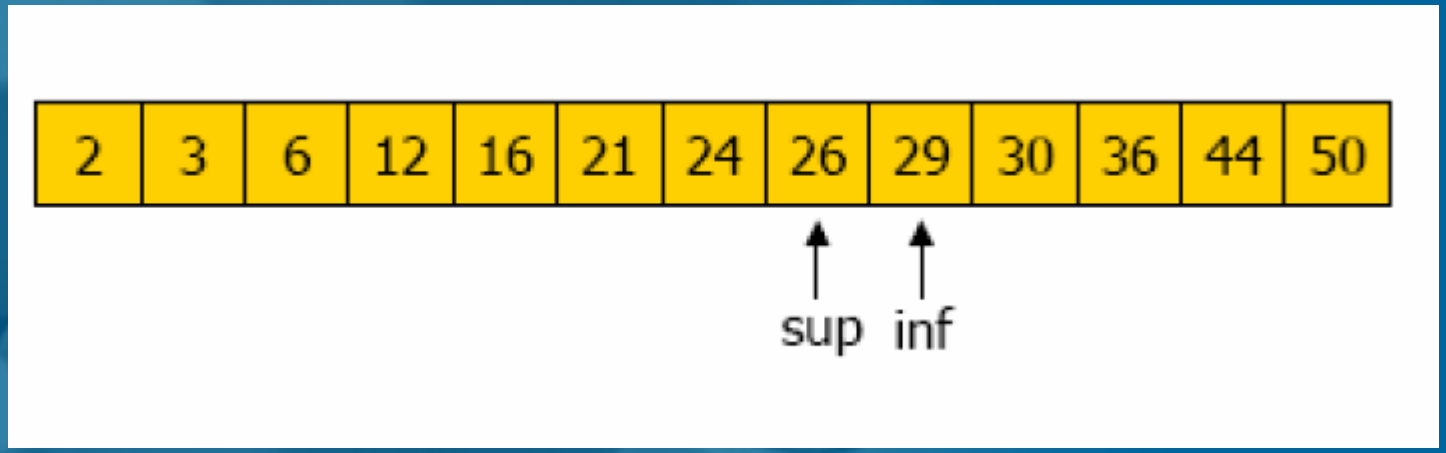
Copyright © 2008 - The McGraw - Hill Companies, srl

Esempio2: ricerca binaria



Copyright © 2008 - The McGraw - Hill Companies, srl

Esempio2: ricerca binaria



Elemento (28) non trovato!

Copyright © 2008 - The McGraw - Hill Companies, srl



Esempio 2 (1/2)

Ricerca binaria di un elemento in un array ordinato

algoritmo `ricercaBinariaIter`$(array\ L,\ elem\ x) \rightarrow booleano$

1. $a \leftarrow 1$
2. $b \leftarrow$ lunghezza di L
3. **while** $(\ L[(a+b)/2] \neq x\ )$ **do**
4. $\quad m \leftarrow (a+b)/2$
5. $\quad$ **if** $(\ L[m] > x\ )$ **then** $b \leftarrow m-1$
6. $\quad$ **else** $a \leftarrow m+1$
7. $\quad$ **if** $(\ a > b\ )$ **then return** non trovato
8. **return** trovato

Si assuma ⌜(a+b)/2⌝

Copyright © 2008 - The McGraw - Hill Companies, srl



Esempio 2 (2/2)

$T_{best}(n) = 1$ (costante) l'elemento centrale è uguale a x

$T_{worst}(n) = O(\lg n)$ $x \notin L$ oppure viene trovato all'ultimo confronto

Quante iterazioni nel ciclo while? Poiché la dimensione del sotto-array su cui si procede si dimezza dopo ogni confronto, dopo l'i-esimo confronto il sottoarray di interesse ha dimensione $n/2^i$.
L'analisi assume n potenza del 2.

$T_{avg}(n) = \lg n - 1 + 1/n$ assumendo che le istanze siano equidistribuite

[Vedi Cap2 pag. 35 del libro Demetrescu et al.]

Copyright © 2008 - The McGraw - Hill Companies, srl



Riepilogo

- Esprimiamo la quantità di una certa risorsa di calcolo (tempo, spazio) usata da un algoritmo in funzione della dimensione n dell'istanza di ingresso
- La notazione asintotica permette di esprimere la quantità di risorsa usata dall'algoritmo in modo sintetico, ignorando dettagli non influenti
- A parità di dimensione n, la quantità di risorsa usata può essere diversa, da cui la necessità di analizzare il caso peggiore o, se possibile, il caso medio
- La quantità di risorsa usata da algoritmi ricorsivi può essere espressa tramite relazioni di ricorrenza, risolvibili tramite vari metodi generali [Prossima lezione]

Copyright © 2008 - The McGraw - Hill Companies, srl